

Návrh a implementace distribuovaného systému pro správu aktiv s využitím Cloud-Native principů

Design and Implementation of a Distributed Asset Management System Using Cloud-Native Principles

Bc. Jan Bojko

Semestrální projekt

Vedoucí projektu: Ing. Radoslav Fasuga, Ph.D.

Ostrava, 2026

Zadání semestrálního projektu

Název: Návrh a implementace distribuovaného systému pro správu aktiv s využitím Cloud-Native principů

Rok zadání: 2025/2026

Vedoucí: Ing. Radoslav Fasuga, Ph.D.

Student: Bc. Jan Bojko

Zaměření: Softwarové Inženýrství

Forma studia: prezenční

Text zadání:

Cílem projektu je návrh a implementace škálovatelného distribuovaného systému pro real-time sledování a lifecycle management fiktivního podnikového majetku (Asset Tracking). Projekt je zaměřen na využití moderní Cloud-Native architektury, asynchronní komunikace mezi službami a automatizaci nasazení do kontejnerizovaného prostředí, včetně vizualizace dat v klientské aplikaci.

1. Student se seznámí s principy Cloud-Native architektury, návrhovým vzorem Microservices a ekosystémem jazyka Go (Golang) pro vývoj backendových služeb.
2. Student provede analýzu požadavků na distribuovaný systém a porovná vhodné komunikační protokoly (gRPC, REST) a metody pro zajištění konzistence dat (Event-Driven Architecture).
3. Student provede implementaci backendové části systému s využitím „Clean Architecture“, včetně kontejnerizace aplikací (Docker) a orchestrace (Kubernetes). Součástí řešení bude i zajištění observability (logování, metriky).
4. Výsledkem bude funkční prototyp distribuovaného systému zahrnující klientskou dashboard aplikaci pro vizualizaci dat, sada automatizovaných testů a také technická příručka popisující: architekturu, lokální nasazení (Kubernetes/Docker) a specifikaci API rozhraní.

Literatura:

1. Donovan, A. A., & Kernighan, B. W. (2015). *The Go Programming Language*. Addison-Wesley Professional. ISBN-13 978-0134190440
2. Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media. ISBN-13 978-1492034025
3. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall. ISBN-13 978-0134494166
4. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media. ISBN-13 978-1491929124
5. Luksa, M. (2017). *Kubernetes in Action*. Manning Publications. ISBN-13 978-1617293726

Abstrakt

Cílem této práce je návrh a implementace distribuovaného systému pro správu firemních aktiv, který je postaven na principech Cloud-Native architektury. Výsledné řešení je tvořeno čtyřmi Go mikroslužbami komunikujícími přes gRPC a NATS JetStream, klientskou dashboard aplikací postavenou na frameworku Next.js a kompletní infrastrukturou definovanou kódem. Práce je zaměřena na praktické uplatnění návrhových vzorů Clean Architecture, Event-Driven Architecture, CQRS a Dependency Injection. Pro orchestraci je využito kontejnerizační platformy Kubernetes ve formě odlehčené distribuce k3s spouštěné nástrojem k3d, doplněné o Helm pro instalaci infrastrukturních komponent, Terraform pro deklarativní provisionování a ArgoCD s Tektonem pro GitOps a kontinuální nasazování. Observabilita je zajištěna pomocí OpenTelemetry, Prometheus, Grafana, Tempo a Loki. Pro ověření funkcionality byla implementována sada 72 automatizovaných testů, které jsou součástí CI pipeline.

Klíčová slova: Cloud-Native; mikroslužby; Go; gRPC; NATS JetStream; Clean Architecture; Event-Driven; CQRS; Kubernetes; k3s; k3d; Docker; Helm; Terraform; ArgoCD; Tekton; OpenTelemetry; Prometheus; Grafana; Tempo; Loki; Next.js; TypeScript; PostgreSQL

Abstract

The goal of this work is the design and implementation of a distributed asset management system built on Cloud-Native architectural principles. The resulting solution is composed of four Go microservices communicating through gRPC and NATS JetStream, a client dashboard application built on the Next.js framework, and a complete infrastructure defined as code. The work is focused on the practical application of the Clean Architecture, Event-Driven Architecture, CQRS, and Dependency Injection design patterns. For orchestration, the Kubernetes container platform is used in the form of the lightweight k3s distribution launched by the k3d tool, together with Helm for installing infrastructure components, Terraform for declarative provisioning, and ArgoCD with Tekton for GitOps and continuous deployment. Observability is provided by OpenTelemetry, Prometheus, Grafana, Tempo, and Loki. A suite of 72 automated tests, which are part of the CI pipeline, has been implemented to verify functionality.

Keywords: Cloud-Native; microservices; Go; gRPC; NATS JetStream; Clean Architecture; Event-Driven; CQRS; Kubernetes; k3s; k3d; Docker; Helm; Terraform; ArgoCD; Tekton; OpenTelemetry; Prometheus; Grafana; Tempo; Loki; Next.js; TypeScript; PostgreSQL

Poděkování

Rád bych zde poděkoval vedoucímu semestrálního projektu Ing. Radoslavu Fasugovi, Ph.D. za odborné vedení a cenné rady během tvoření tohoto semestrálního projektu. Dále děkuji Fakultě elektrotechniky a informatiky VŠB, Technické univerzity Ostrava za poskytnuté zázemí a podporu při studiu.

Obsah

1	Úvod	8
2	Architektonické vzory a principy	9
2.1	Cloud-Native architektura	9
2.2	Microservices	9
2.3	Clean Architecture	9
2.4	Event-Driven Architecture	10
2.5	CQRS (Command Query Responsibility Segregation)	11
2.6	Dependency Injection	11
2.7	12-Factor App	12
3	Použité technologie	13
3.1	Go (Golang)	13
3.2	gRPC a Protocol Buffers	13
3.3	NATS JetStream	13
3.4	PostgreSQL a pgx	13
3.5	chi router	14
3.6	Next.js	14
3.7	TypeScript, SWR, Zod, Leaflet	14
3.8	Docker	14
3.9	Kubernetes (k3s, k3d)	14
3.10	Helm	14
3.11	Terraform	15
3.12	ArgoCD	15
3.13	Tekton	15
3.14	OpenTelemetry	15
3.15	Prometheus a Grafana	15
3.16	Grafana Tempo	16
3.17	Loki a Promtail	16
4	Návrh systému	17
4.1	Celková architektura	17
4.2	Datové toky	17
4.3	Životní cyklus assetu	17
5	Implementace	19
5.1	Doménová vrstva	19
5.2	gRPC adaptér	19
5.3	NATS klient a propagace trace kontextu	19
5.4	Byznys logika v Processing Service	20
5.5	HTTP adaptér v Query Service	21
5.6	Simulator	21
5.7	Klientská aplikace	22

6	Nasazení a infrastruktura	24
6.1	Infrastruktura jako kód s Terraformem	24
6.2	Kubernetes manifesty	24
6.3	GitOps s ArgoCD	25
6.4	Tekton pipeline	26
6.5	Vícefázový Dockerfile	27
7	Observabilita	28
7.1	Metriky	28
7.2	Distribuované trasování	28
7.3	Logy	29
8	Testování	30
9	Závěr	30

Seznam obrázků

1	Klientská dashboard aplikace s mapou pozice lodi a panelem telemetrie	23
2	Webové rozhraní ArgoCD se stavem synchronizace aplikace	26
3	Vlastní aplikační metriky vizualizované v Grafaně	28
4	Distribuovaná trace zachycena v Grafana Tempu	29

Seznam listingů (ukázek kódu)

1	Portová rozhraní pro repozitáře (<code>backend/internal/core/ports/repository.go</code>)	10
2	Kompozice závislostí ve službě processor (<code>backend/cmd/processor/main.go</code>) . .	11
3	Publikace zprávy do NATS s injektovaným trace kontextem	19
4	Vyhodnocení obchodních pravidel ve službě processor	20
5	SWR hooky pro pravidelné stahování dat (<code>frontend/hooks/useAssets.ts</code>) . . .	22
6	Definice Helm releasu pro NATS JetStream (<code>infrastructure.tf</code>)	24
7	Tekton Task pro build a push obrazu pomocí Kaniko (<code>task-build-push.yaml</code>) .	26

Seznam použitých zkratek a symbolů

Zkratka	Význam
API	Application Programming Interface (Aplikační programové rozhraní)
CDN	Content Delivery Network (Síť pro doručování obsahu)
CI	Continuous Integration (Kontinuální integrace)
CNCF	Cloud Native Computing Foundation
CPU	Central Processing Unit (Procesor)
CQRS	Command Query Responsibility Segregation
DNS	Domain Name System (Systém doménových jmen)
GHC	GitHub Container Registry
GPS	Global Positioning System (Globální polohovací systém)
gRPC	gRPC Remote Procedure Calls
HTTP	Hypertext Transfer Protocol (Hypertextový přenosový protokol)
JSON	JavaScript Object Notation (Zápis objektů v jazyce JavaScript)
NATS	Neural Autonomic Transport System (Message broker)
OTLP	OpenTelemetry Protocol
PCB	Printed Circuit Board (Plošný spoj)
REST	Representational State Transfer (Přenos reprezentačního stavu)
RH	Relative Humidity (Relativní vlhkost)
SDK	Software Development Kit (Vývojářská sada)
SHA	Secure Hash Algorithm (Bezpečný hashovací algoritmus)
SSG	Static Site Generation (Vykreslování statických stránek)
SSR	Server-Side Rendering (Vykreslování na straně serveru)
SVG	Scalable Vector Graphics (Škálovatelná vektorová grafika)
SWR	Stale-While-Revalidate
UI	User Interface (Uživatelské rozhraní)
URL	Uniform Resource Locator (Jednotný lokátor zdrojů)
W3C	World Wide Web Consortium
YAML	YAML Ain't Markup Language (Konfigurační formát)

1 Úvod

Moderní distribuované systémy jsou v současné době nejen tématem akademické literatury, ale především průmyslovým standardem, na kterém je postavena většina aplikací v prostředí komerčních cloudových služeb. Důvodem je především škálovatelnost, odolnost vůči výpadkům a možnost rychlého dodávání nových funkcí do produkčního prostředí. S rozšířením kontejnerizačních technologií a orchestračních platforem se tento přístup stal dostupným i pro menší týmy programátorů, a to bez nutnosti budovat vlastní datová centra či mít vlastní servery.

Cílem tohoto semestrálního projektu je navržení a implementace funkčního prototypu distribuovaného systému pro správu firemních aktiv (či assetů), který demonstruje praktické využití Cloud-Native principů [1]. Doménou projektu je sledování lodních kontejnerů přepravujících citlivý průmyslový náklad, u kterého je nutné kontinuálně monitorovat polohu, teplotu, vlhkost a stav uzamčení. Pro účely simulace byl zvolen scénář dovozu niobových terčů a PCB substrátů z Jižní Koreje pro výrobce supravodivých kvantových počítačů, přičemž samotná plavba je modelována na trase Pusan => Rotterdam.

V rámci tohoto projektu byly aplikovány návrhové vzory Clean Architecture, Event-Driven Architecture, CQRS a Dependency Injection, a to v kontextu jazyka Go a ekosystému Kubernetes. Asynchronní komunikace mezi mikroslužbami je zajištěna pomocí NATS JetStream, synchronní komunikace pak prostřednictvím gRPC. Pro klientskou dashboard aplikaci byl zvolen framework Next.js spolu s knihovnami React, TypeScript, SWR a Leaflet.

Celá infrastruktura byla popsána jako kód (Infrastructure as Code) pomocí nástroje Terraform a Helm chartů. Kontinuální nasazování je řešeno přístupem GitOps prostřednictvím ArgoCD, sestavování obrazů a spouštění testů je zajištěno nástrojem Tekton. Observabilita systému je založena na nástrojích OpenTelemetry, Prometheus, Grafana, Tempo a Loki, čímž jsou pokryty všechny tři pilíře pozorovatelnosti, tedy metriky, distribuované trasování a logy.

Práce je strukturována tak, aby čtenáře nejprve seznámila s použitými architektonickými vzory a technologiemi, následně byl popsán návrh systému a jeho implementace, a v závěrečných kapitolách bylo představeno nasazení do clusteru, observabilita a strategie testování.

2 Architektonické vzory a principy

Tato kapitola představuje hlavní návrhové vzory a principy, na kterých je projekt postaven. Každý z popsaných vzorů je v implementaci přímo využit a v následujících kapitolách jsou uvedeny konkrétní příklady jeho použití.

2.1 Cloud-Native architektura

Termínem Cloud-Native je označován přístup k tvorbě a provozu aplikací, který je primárně určen pro běh v dynamických prostředích veřejných, soukromých či hybridních cloudů. Aplikace navrhované tímto způsobem jsou typicky tvořeny menšími, nezávisle nasaditelnými komponentami, které jsou kontejnerizovány, orchestrovány a automaticky škálovány na základě skutečného zatížení [1]. Klíčovými rysy jsou deklarativní konfigurace, neměnnost infrastruktury, automatizace nasazení a odolnost vůči částečným výpadkům.

V tomto projektu jsou Cloud-Native principy použity v několika vrstvách. Aplikační vrstva je rozdělena do čtyř nezávislých mikroslužeb, z nichž každá běží ve vlastním Docker kontejneru a je nasazena v Kubernetes clusteru. Infrastruktura je popsána deklarativně pomocí Terraformu a Helm chartů. Image kontejnerů jsou neměnné, identifikované krátkým Git SHA, a aktualizace produkčního prostředí probíhá přes GitOps workflow.

2.2 Microservices

Architektonický vzor Microservices (Mikro-slужby) spočívá v rozdělení monolitické aplikace na sadu menších, nezávislých služeb, z nichž každá je zaměřena na jednu businessovou logiku a je nasaditelná samostatně [2]. Komunikace mezi službami probíhá typicky přes lehké protokoly, jako jsou HTTP REST, gRPC nebo asynchronní zprávy v rámci message brokeru.

V rámci tohoto projektu byly naprogramovány čtyři mikroslužby napsané v jazyce Go, které se nacházejí v adresáři `backend/cmd/`. Konkrétně se jedná o službu `ingest` pro příjem telemetrie přes gRPC, `processor` pro asynchronní zpracování zpráv z fronty, `query` pro obsluhu dotazů z klientské aplikace a `simulator`, který generuje syntetickou telemetrii pro 50 simulovaných kontejnerů. Pátou komponentou je klientská aplikace `frontend` postavená na frameworku Next.js. Každá z těchto služeb má vlastní vstupní bod `main.go`, vlastní Docker image, vlastní Kubernetes Deployment a vlastní sadu Service objektů.

2.3 Clean Architecture

Clean Architecture je sada zásad pro strukturování zdrojového kódu, kterou popsal Robert C. Martin v jeho knize Clean Architecture [3]. Hlavní myšlenkou tohoto přístupu je, že závislosti mezi jednotlivými vrstvami aplikace musí směřovat zevnitř ven, čímž je doménová vrstva izolována od konkrétních infrastrukturních detailů, jako jsou databáze, message brokery nebo komunikační protokoly.

V tomto projektu byla Clean Architecture aplikována formou rozdělení backendového kódu do tří hlavních vrstev. Doménová vrstva, která se nachází v adresáři `backend/internal/core/domain`, obsahuje business entity `Asset`, `TelemetryData` a `Alert` bez jakékoliv vnější závislosti. Vrstva portů v adresáři `backend/internal/core/ports` definuje

rozhraní (Go `interface`), která jsou doménou požadována pro komunikaci s vnějším světem. Konečně vrstva adaptérů v adresáři `backend/internal/adapters` implementuje tato rozhraní pomocí konkrétních technologií, jako jsou PostgreSQL, NATS JetStream, gRPC a HTTP.

Listing 1 obsahuje příklad portového rozhraní, které je doménou definováno a adaptéry implementováno.

```
1 package ports
2
3 import (
4     "context"
5     "github.com/BojkoJ/go-nextjs-smart-tracking-system/backend/internal/core/
        domain"
6 )
7
8 type AssetRepository interface {
9     SaveAsset(ctx context.Context, asset domain.Asset) error
10    GetAssetByID(ctx context.Context, assetID string) (*domain.Asset, error)
11    ListAssets(ctx context.Context) ([]domain.Asset, error)
12    UpdateAssetStatus(ctx context.Context, assetID string, status domain.
        AssetStatus) error
13 }
14
15 type TelemetryRepository interface {
16     SaveTelemetry(ctx context.Context, telemetry domain.TelemetryData) error
17     GetLastTelemetry(ctx context.Context, assetID string) (*domain.TelemetryData
        , error)
18     ListTelemetryHistory(ctx context.Context, assetID string, limit, offset int)
        ([]domain.TelemetryData, error)
19 }
20
21 type AlertRepository interface {
22     SaveAlert(ctx context.Context, alert domain.Alert) error
23     ListAlerts(ctx context.Context) ([]domain.Alert, error)
24     ListAllAlertsForAsset(ctx context.Context, assetID string) ([]domain.Alert,
        error)
25 }
```

Listing 1: Portová rozhraní pro repozitáře (`backend/internal/core/ports/repository.go`)

Konkrétní implementace těchto rozhraní pro PostgreSQL je obsažena v souboru `backend/internal/adapters/repository/postgres_repo.go`. Vrstva služeb (use cases) je v adresáři `backend/internal/services` a obsahuje business logiku, která je nezávislá na konkrétních technologiích.

2.4 Event-Driven Architecture

Architektura řízená událostmi (Event-Driven Architecture) je vzor, ve kterém je komunikace mezi komponentami zprostředkována publikací a konzumací událostí, typicky prostřednictvím

message brokeru [4]. Hlavními výhodami jsou volné spřažení komponent, lepší odolnost vůči výpadkům a možnost asynchronního zpracování velkých objemů dat.

V tomto projektu je Event-Driven Architecture realizována pomocí NATS JetStream. Služba `ingest` publikuje zprávy do streamu `TELEMETRY` na subjekt `telemetry.ingest`, služba `processor` se na tento subjekt přihlašuje jako trvalý (durable) konzument. Důsledkem tohoto rozdělení je, že pokud služba `processor` selže nebo je restartována, zprávy zůstávají uloženy v JetStream frontě a jsou doručeny po obnovení činnosti, čímž je zajištěna garance doručení alespoň jednou (at-least-once delivery).

2.5 CQRS (Command Query Responsibility Segregation)

Vzor CQRS spočívá v oddělení modelu pro zápis dat od modelu pro čtení dat [5]. Místo jednoho monolitického repozitáře, který obsluhuje obě cesty, jsou v CQRS systému vytvořeny dva samostatné toky, které mohou být škálovány a optimalizovány nezávisle.

V tomto projektu byl zvolen zjednodušený CQRS přístup. Příkazový tok (Command) je realizován posloupností `simulator` → gRPC → `ingest` → NATS → `processor` → PostgreSQL. Dotazovací tok (Query) je realizován posloupností `frontend` → HTTP → `query` → PostgreSQL. Tímto je dosaženo toho, že čtení neblokuje zápis a obě cesty mohou být škálovány samostatně, což je v praxi využito horizontálním autoškálováním služby `query` (manifest `hpa.yaml`).

2.6 Dependency Injection

Dependency Injection je technika, ve které jsou závislosti komponenty (typicky implementace rozhraní) předávány zvenku, místo aby si je komponenta vytvářela sama. Důsledkem je lepší testovatelnost, jelikož v testech mohou být skutečné implementace nahrazeny mocky či faky.

V tomto projektu je každá funkce `main()` kompozičním kořenem, ve kterém jsou vytvářeny konkrétní implementace (NATS klient, PostgreSQL repozitář) a předávány jako rozhraní (`ports.EventProducer`, `ports.AssetRepository`, ...) do služeb a handlerů. Listing 2 ukazuje příklad kompozice ve službě `processor`.

```
1 pgRepo, err := repository.NewPostgresRepo(context.Background(), config.
    PostgresURL)
2 if err != nil {
3     logger.Error("failed to create new postgres repo", "error", err)
4     os.Exit(1)
5 }
6
7 // pgRepo implementuje 3 rozhrani (AssetRepository, TelemetryRepository,
    AlertRepository)
8 processingService := services.NewProcessingService(pgRepo, pgRepo, pgRepo,
    logger)
9
10 natsClient, err := eventbus.NewNATSClient(config.NATSUrl, logger)
11 if err != nil {
12     logger.Error("failed to create new NATS client", "error", err)
13     os.Exit(1)
```

Listing 2: Kompozice závislostí ve službě processor (`backend/cmd/processor/main.go`)

2.7 12-Factor App

Metodika 12-Factor App je sada doporučení pro tvorbu aplikací určených pro provoz v cloudu [6]. Z této metodiky jsou v projektu uplatněny především následující principy. Konfigurace je výhradně předávána pomocí proměnných prostředí (princip III), což je realizováno funkcí `common.Load()`, která validuje povinné proměnné `NATS_URL` a `POSTGRES_URL`. Žádné konfigurační hodnoty nejsou pevně vloženy do obrazu kontejneru. Závislosti jsou explicitně deklarovány v souboru `go.mod` (princip II). Procesy aplikace jsou bezstavové (princip VI), veškerý stav je uložen v PostgreSQL nebo v NATS JetStream. Logy jsou zapisovány na standardní výstup ve strukturovaném JSON formátu (princip XI), což je dále využito agentem Promtail.

3 Použité technologie

Tato kapitola obsahuje stručný popis jednotlivých technologií, které byly v projektu nasazeny. U každé technologie je uvedeno, jakou úlohu v projektu plní a kterých souborů či adresářů se týká.

3.1 Go (Golang)

Go je staticky typovaný kompilovaný jazyk vyvinutý společností Google, který je navržen s důrazem na jednoduchost, výkon a nativní podporu paralelismu [7]. V projektu byl Go zvolen jako základ pro všechny čtyři backendové mikroslužby. Verze 1.25 byla použita jak v lokálním vývoji, tak v build fázi Dockerfile. Veškerá zdrojová sada Go souborů se nachází v adresáři `backend/`, vstupní body jednotlivých mikroslužeb pak v podadresářích `backend/cmd/`.

3.2 gRPC a Protocol Buffers

gRPC je framework pro vzdálené volání procedur založený na HTTP/2 a na binárním kódování Protocol Buffers [8]. V projektu je gRPC využit pro synchronní komunikaci mezi službami `simulator` a `ingest`. Kontrakt `TelemetryService` je definován v souboru `backend/proto/telemetry.proto` a vygenerovaný Go kód je umístěn v souborech `telemetry.pb.go` a `telemetry_grpc.pb.go`. Volbou gRPC byla získána výrazně menší velikost přenášených zpráv (přibližně pětikrát menší než ekvivalentní JSON přes HTTP) a automatické přidělování každého příchozího volání do vlastní goroutiny, čímž je dosaženo paralelního zpracování bez nutnosti psát kód pro synchronizaci.

3.3 NATS JetStream

NATS je vysokovýkonný message broker s modelem publish-subscribe [9]. Rozšíření JetStream přidává perzistentní úložiště zpráv s podporou trvanlivých konzumentů, opětovného doručení a filtru subjektů. V projektu je NATS JetStream nasazen v namespace `infrastructure` pomocí Helm chartu `nats`, instalovaného Terraformem (soubor `backend/deploy/terraform/infrastructure.tf`). Stream s názvem `TELEMETRY` a subjekty `telemetry.>` je vytvářen aplikačním kódem v souboru `backend/internal/adapters/eventbus/nats_client.go` při startu klienta.

3.4 PostgreSQL a pgx

PostgreSQL je relační databázový systém, který je v projektu využit pro perzistentní uložení assetů, telemetrie a alertů. Schéma databáze je vytvářeno Kubernetes Jobem `db-init`, který je spouštěn ArgoCD jako PreSync hook (soubor `backend/deploy/k8s/db-init-job.yaml`). Pro připojení z Go kódu byla zvolena knihovna `pgx/v5`, která je nativním driverem pro PostgreSQL bez nutnosti používat bridge přes `database/sql`. Tím je dosaženo přibližně dvojnásobné až trojnásobné rychlosti oproti standardní knihovně a podpory PostgreSQL-specifických typů. Pool spojení je vytvářen funkcí `pgxpool.New()` a používán v souboru `backend/internal/adapters/repository/postgres_repo.go`.

3.5 chi router

chi je odlehčený HTTP router pro Go, kompatibilní se standardní knihovnou `net/http`. V projektu je použit ve službě `query` pro směrování REST endpointů a pro získávání URL parametrů (soubor `backend/internal/adapters/handler/http_handler.go`).

3.6 Next.js

Next.js je framework pro tvorbu React aplikací s podporou serverového vykreslování (SSR) a generování statických stránek (SSG) [10]. V projektu byla použita verze 16 spolu s Reactem ve verzi 19. Klientská aplikace je umístěna v adresáři `frontend/` a obsahuje routovací strukturu v adresáři `app/`, sdílené komponenty v adresáři `components/` a hooky pro práci s daty v adresáři `hooks/`.

3.7 TypeScript, SWR, Zod, Leaflet

TypeScript byl ve frontendové části zvolen pro statickou typovou kontrolu. Knihovna SWR je využita pro pravidelné stahování dat z REST API s automatickou revalidací, knihovna Zod pro runtime validaci JSON odpovědí a odvození TypeScript typů (soubor `frontend/lib/schemas.ts`). Pro vykreslení interaktivní mapy s polohou lodi byla použita knihovna Leaflet, která je dynamicky načítána z CDN v komponentě `ShipMap` (soubor `frontend/components/ShipMap.tsx`).

3.8 Docker

Docker je platforma pro tvorbu a běh kontejnerů. V projektu je využita pro sestavování obrazů všech backendových mikroslužeb i klientské aplikace. Společný vícefázový Dockerfile pro Go služby je umístěn v souboru `backend/Dockerfile`. Volbou parametru `ARG SERVICE` je rozhodnuto, která ze čtyř mikroslužeb je v daném buildu kompilována. Tím je zachována jediná zdrojová definice obrazu pro všechny služby, čímž je redukována udržovací zátěž.

3.9 Kubernetes (k3s, k3d)

Kubernetes je orchestrační platforma pro správu kontejnerizovaných aplikací [11]. Pro lokální vývoj byla zvolena odlehčená distribuce `k3s` od společnosti Rancher, která je doručována ve formě jediné binárky o velikosti přibližně 60 MB. Distribuce `k3s` obsahuje výchozí Traefik Ingress controller a je optimalizována pro prostředí s omezenými prostředky. Nástrojem `k3d` je každý uzel clusteru zabalen do Docker kontejneru, čímž je celý cluster spuštěn na jediném vývojářském počítači. Topologie clusteru je tvořena jedním control-plane uzlem, dvěma worker uzly a load balancerem.

3.10 Helm

Helm je balíčkovací systém pro Kubernetes, který umožňuje instalovat a udržovat aplikace ve formě Helm chartů. V projektu je Helm využit pro instalaci infrastrukturních komponent jako NATS, PostgreSQL, kube-prometheus-stack, Grafana Tempo a Loki-stack. Tyto Helm releasy jsou deklarovány v Terraformu (soubor `backend/deploy/terraform/infrastructure.tf`) a jsou instalovány automaticky při spuštění příkazu `terraform apply`.

3.11 Terraform

Terraform je nástroj pro deklarativní popis infrastruktury (Infrastructure as Code). V projektu je Terraformem zajištěno zprostředkování celého k3d clusteru, vytvoření Kubernetes namespace a instalace Helm chartů s infrastrukturními komponentami. Terraform soubory se nachází v adresáři `backend/deploy/terraform/` a jsou organizovány do souborů `main.tf`, `cluster.tf`, `namespaces.tf`, `infrastructure.tf`, `argocd.tf` a `variables.tf`. Použité providery jsou `moio/k3d`, `hashicorp/kubernetes` a `hashicorp/helm`.

3.12 ArgoCD

ArgoCD je nástroj pro GitOps kontinuální nasazování, který průběžně srovnává stav clusteru s deklarovaným stavem v Git repozitáři. V projektu je ArgoCD nasazen v namespace `argocd` pomocí Helm chartu (soubor `backend/deploy/terraform/argocd.tf`). ArgoCD aplikace je definována v souboru `backend/deploy/argocd/application.yaml` a je nakonfigurována tak, aby sledovala adresář `backend/deploy/k8s` na větvi `master`. Při detekci změny manifestů je provedena automatická synchronizace s politikou `prune: true` a `selfHeal: true`.

3.13 Tekton

Tekton je framework pro deklarativní definici CI pipeline, který běží přímo uvnitř Kubernetes clusteru [12]. V projektu je Tekton využit pro sestavování Docker obrazů, spouštění testů a aktualizaci image tagů v deployment manifestech. Tekton zdroje se nachází v adresáři `backend/deploy/tekton/` a jsou členěny na Task objekty (`task-git-clone.yaml`, `task-run-tests.yaml`, `task-build-push.yaml`, `task-update-manifest.yaml`), na Pipeline objekt (`pipeline.yaml`) a na PipelineRun objekt (`pipeline-run.yaml`). Pro sestavování obrazů uvnitř clusteru je použit nástroj Kaniko, kterým jsem se vyhnul nutnosti přístupu k Docker daemonu.

3.14 OpenTelemetry

OpenTelemetry je sada SDK a API pro instrumentaci aplikací, která je standardizována pod hlavičkou CNCF [13]. V projektu je OpenTelemetry využita pro generování distribuovaných trace spanů ve všech čtyřech Go službách. Inicializace je provedena funkcí `InitTracerProvider()` v souboru `backend/internal/common/otel.go`. Trace kontext je propagován standardem W3C `TraceContext`, a to jak přes hranici gRPC volání pomocí `otelgrpc`, tak přes hranici NATS zpráv pomocí vlastního `TextMapCarrier` (soubor `nats_client.go`).

3.15 Prometheus a Grafana

Prometheus je systém pro sběr metrik formou periodického scrapování HTTP endpointů. V projektu je Prometheus nasazen jako součást chartu `kube-prometheus-stack`. Vlastní aplikační metriky jsou definovány v souboru `backend/internal/common/metrics.go` a jsou vystaveny na cestě `/metrics`. Prometheus Operator automaticky detekuje `ServiceMonitor` objekty (soubor `backend/deploy/k8s/servicemonitor.yaml`) s labelem `release: kube-prometheus-stack`

a začne dané služby pravidelně scrapovat. Grafana je nasazena spolu s Prometheem a poskytuje grafické rozhraní pro vizualizaci metrik a prohlížení trasování i logů.

3.16 Grafana Tempo

Tempo je distribuovaný backend pro ukládání trace spanů. V projektu je Tempo nakonfigurován tak, aby přijímal OTLP zprávy přes gRPC na portu 4317, kam jsou tracingová data odesílána ze všech čtyř Go služeb. Konfigurace je provedena v Terraformu (soubor `backend/deploy/terraform/infrastructure.tf`, blok `helm_release.temp`).

3.17 Loki a Promtail

Loki je systém pro agregaci a indexaci logů, který je v projektu nasazen společně s agentem Promtail jako součást chartu `loki-stack`. Promtail běží jako DaemonSet na každém uzlu clusteru a čte logy z cest `/var/log/pods`, které jsou následně odesílány do Loki. Strukturované JSON logy zapisované Go službami pomocí knihovny `log/slog` jsou tak automaticky indexovány a v Grafaně mohou být filtrovány podle pole `service`.

4 Návrh systému

Tato kapitola popisuje celkovou architekturu systému a datové toky mezi jednotlivými komponentami. Konkrétní implementační detaily jsou rozpracovány v následující kapitole.

4.1 Celková architektura

Systém je tvořen pěti hlavními aplikačními komponentami, čtyřmi backendovými mikroslužbami napsanými v jazyce Go a jednou klientskou aplikací postavenou na frameworku Next.js. Tyto komponenty jsou nasazeny v Kubernetes namespace `tracking-system`. Infrastrukturní komponenty, tedy NATS JetStream a PostgreSQL, jsou nasazeny v namespace `infrastructure`. Komponenty observability stacku jsou nasazeny v namespace `monitoring`. ArgoCD je nasazeno v namespace `argocd`.

Architektura systému je popsána následujícím schématem datových toků:

```
Simulator --gRPC--> Ingest --> NATS JetStream --> Processor --> PostgreSQL
                                                                    |
Frontend <--HTTP REST----- Query <-----+----->
```

4.2 Datové toky

V systému jsou rozlišeny dva základní datové toky, které odpovídají rozdělení dle vzoru CQRS popsanému v kapitole 2.5.

Příkazový tok je realizován posloupností `simulator → ingest → NATS JetStream → processor → PostgreSQL`. V tomto směru jsou každou sekundu generovány telemetrické záznamy pro každý z 50 simulovaných kontejnerů, čímž je vytvořena zátěž přibližně 50 gRPC volání za sekundu. Služba `ingest` pouze validuje vstupní data, serializuje je do JSON formátu a odesílá do NATS JetStream fronty bez čekání na zpracování. Tímto je dosaženo velmi nízké latence příjmu, která není závislá na rychlosti zpracování. Služba `processor` pak asynchronně konzumuje zprávy z fronty, vyhodnocuje obchodní pravidla a ukládá data do PostgreSQL.

Dotazovací tok je realizován posloupností `frontend → Traefik Ingress → query → PostgreSQL`. Klientská aplikace každých 5 až 10 sekund stahuje aktuální stav assetů, poslední telemetrický záznam a historii pro vybraný kontejner. Služba `query` čte data přímo z PostgreSQL bez zapojení NATS.

Tok observability je doplňkový a probíhá souběžně se dvěma předchozími. Veškeré metriky jsou periodicky scrapovány Prometheem z endpointu `/metrics`, trace spany jsou exportovány přes OTLP gRPC do Tempa a strukturované logy jsou agentem Promtail sbírány a odesílány do Loki.

4.3 Životní cyklus assetu

Každý ze 50 simulovaných lodních kontejnerů (asset) prochází v rámci své doby trvání stavovým automatem. V doménové vrstvě (soubor `backend/internal/core/domain/asset.go`) jsou definovány čtyři možné stavy. Stav `new` reprezentuje nově registrovaný kontejner, který dosud nebyl aktivován. Stav `active` (zkratkou též označovaný jako OK) reprezentuje běžně provozovaný kontejner, jehož telemetrie je v rámci stanovených limitů. Stav `maintenance` reprezentuje kontejner,

u něhož byla detekována nestandardní situace vyžadující servisní zásah. Stav **decommissioned** reprezentuje kontejner, který byl vyřazen z provozu, typicky po doručení do cílové destinace.

Při inicializaci databáze (Kubernetes Job **db-init**) je každému z 50 assetů přiřazen výchozí stav **active**, výchozí limit maximální teploty 28 °C, minimální teploty 12 °C a maximální vlhkosti 45 % RH. Tyto limity reprezentují skutečné požadavky pro přepravu citlivého průmyslového nákladu, konkrétně niobových terčů a PCB substrátů určených pro výrobu supravodivých kvantových počítačů.

V aktuální implementaci je v souboru **backend/internal/services/processing.go** službou **processor** automaticky řešen jediný přechod, a to z **active** na **decommissioned**. K tomuto přechodu dochází v okamžiku, kdy Haversinova vzdálenost mezi aktuální pozicí lodi a přístavem Rotterdam klesne pod 50 km, čímž je modelováno doručení nákladu do cílové destinace. Stav **maintenance** je v doméně připraven jako rezervovaný cíl pro manuální servisní zásah nebo pro budoucí rozšíření o automatický přechod na základě opakovaného porušení limitů. Přechod do tohoto stavu v současné verzi není automaticky vyvoláván žádným pravidlem v **Processoru**.

Při překročení některého z limitů (teplota nad maximem, teplota pod minimem, vlhkost nad maximem, odemčený zámek) je nezávisle na stavovém automatu vygenerován alert příslušného typu a uložen do tabulky **alerts**.

5 Implementace

V této kapitole jsou popsány klíčové implementační detaily backendových mikroslužeb a klientské aplikace. Pro lepší ilustraci jsou vybrané pasáže kódu uvedeny ve formě listingů.

5.1 Doménová vrstva

Doménová vrstva je středem aplikace a je tvořena business entitami bez jakékoliv vnější závislosti. V souborech `asset.go`, `telemetry.go` a `alert.go` jsou definovány tři hlavní entity systému. Pro entitu `Asset` byly vedle základních atributů (ID, jméno, čas vytvoření) zavedeny obchodní limity přímo v doméně, čímž je vyjádřeno, že tyto hodnoty jsou součástí byznysového kontraktu konkrétního kontejneru, nikoliv konfigurace služby.

Pro reprezentaci stavu assetu byl využit Go idiom pojmenovaného typu nad `string` spolu se sadou konstant, kterými je modelován stavový automat `new` $\rightarrow$ `active` $\rightarrow$ `maintenance` $\rightarrow$ `decommissioned`. Stejným způsobem je definován i typ `AlertType`, kterým jsou kategorizovány jednotlivé typy generovaných alertů.

5.2 gRPC adaptér

Vstupní brána pro telemetrii je tvořena gRPC handlerem v souboru `backend/internal/adapters/handler/grpc_handler.go`. Tento handler implementuje vygenerované rozhraní `TelemetryServiceServer` z protobuf definice a jeho jedinou metodou je `SendTelemetry`. V rámci této metody je provedeno namapování `*pb.TelemetryRequest` na doménovou strukturu `domain.TelemetryData`, čímž je dodržena hranice mezi adaptérem a doménou, za kterou není dovoleno propagovat závislost na vygenerovaném protobuf kódu.

Trace ID je preferenčně získáváno z aktivního OpenTelemetry span contextu, který je do gRPC volání injektován klientem pomocí instrumentace `otelgrpc`. Pokud aktivní span není nalezen (typicky při testech), je použita hodnota z pole `TraceId` zprávy.

5.3 NATS klient a propagace trace kontextu

Implementace NATS klienta je obsažena v souboru `backend/internal/adapters/eventbus/nats_client.go`. Klient implementuje dvě portová rozhraní, `EventProducer` a `EventConsumer`, čímž je umožněno použít jednu instanci ve dvou rolích. Při publikaci zprávy je trace kontext aktivního OpenTelemetry spanu injektován do NATS hlaviček pomocí pomocné struktury `natsHeaderCarrier`, která implementuje rozhraní `propagation.TextMapCarrier`. Při konzumaci zprávy je tento kontext naopak extrahován, čímž je v Grafana Tempu zachována souvislá trace přes hranici asynchronní fronty.

```
1 func (client *NATSClient) PublishMessage(ctx context.Context, subject string,
   data []byte) error {
2     msg := &nats.Msg{
3         Subject: subject,
4         Data:     data,
5         Header:   nats.Header{},
6     }
```

```

7      // Injektuj W3C trace context do NATS message headers (traceparent,
      tracestate)
8      otel.GetTextMapPropagator().Inject(ctx, natsHeaderCarrier{msg.Header})
9
10     _, err := client.JetStreamContext.PublishMsg(msg)
11     if err != nil {
12         return fmt.Errorf("publishing message to JetStream: %w", err)
13     }
14     return nil
15 }

```

Listing 3: Publikace zprávy do NATS s injektovaným trace kontextem

Konzument je registrován jako trvalý (durable) pod identifikátorem `processor-consumer`. Tím je zajištěno, že pokud je služba `processor` restartována, je obnovena z poslední potvrzené pozice ve frontě a žádná zpráva není ztracena.

5.4 Byznys logika v Processing Service

Mozkem celého systému je služba `processor`, která konzumuje zprávy z NATS, ukládá je do PostgreSQL a vyhodnocuje obchodní pravidla. Implementace je obsažena v souboru `backend/internal/services/processing.go`. V Listingu 4 je uveden zkrácený výňatek zachycující jádro této logiky.

```

1 func (s *ProcessingServiceImpl) ProcessTelemetry(ctx context.Context, telemetry
    domain.TelemetryData) error {
2     start := time.Now()
3     defer func() {
4         common.ProcessingDuration.Observe(time.Since(start).Seconds())
5         common.TelemetryProcessed.Inc()
6     }()
7
8     asset, err := s.assetRepo.GetAssetByID(ctx, telemetry.AssetId)
9     if err != nil && asset == nil {
10         return fmt.Errorf("getting an asset by ID: %w", err)
11     }
12     if asset == nil {
13         return nil
14     }
15
16     if err := s.telemetryRepo.SaveTelemetry(ctx, telemetry); err != nil {
17         return fmt.Errorf("saving telemetry: %w", err)
18     }
19
20     if telemetry.Temperature > asset.MaxTemperature {
21         alert := domain.Alert{
22             ID: uuid.New().String(), AssetID: asset.ID,
23             Type:    domain.AlertTemperatureMax,
24             Message: fmt.Sprintf("temperature %.1f exceeded max %.1f", telemetry
                .Temperature, asset.MaxTemperature),

```

```

25         CreatedAt: time.Now(),
26     }
27     if err := s.alertRepo.SaveAlert(ctx, alert); err != nil {
28         return fmt.Errorf("saving alert: %w", err)
29     }
30     common.AlertsGenerated.WithLabelValues(string(domain.AlertTemperatureMax
31     )).Inc()
32     // Analogicky pro MinTemperature, MaxHumidity a IsLocked.
33
34     if asset.Status == domain.StatusActive {
35         distKm := haversine(telemetry.Latitude, telemetry.Longitude,
36         rotterdamLat, rotterdamLon)
37         if distKm <= arrivalThresholdKm {
38             _ = s.assetRepo.UpdateAssetStatus(ctx, asset.ID, domain.
39             StatusDecommissioned)
40         }
41     }
42     return nil
43 }

```

Listing 4: Vyhodnocení obchodních pravidel ve službě processor

Kompletní implementace pokrývá čtyři typy alertů (`temperature_exceeded_max_limit`, `temperature_exceeded_min_limit`, `humidity_exceeded_max_limit`, `container_unlocked`) a přechod stavu na `decommissioned` při dosažení 50 km vzdálenosti od přístavu Rotterdam. Vzdálenost je počítána Haversinovou formulí v pomocné funkci `haversine`, čímž je dosaženo fyzikálně správného výsledku po povrchu Země.

5.5 HTTP adaptér v Query Service

REST API pro klientskou aplikaci je obsluhováno službou `query`, jejíž HTTP handler je implementován v souboru `backend/internal/adapters/handler/http_handler.go`. Pro směrování je použit router `chi`. Bylo registrováno celkem sedm endpointů pro výpis assetů, načítání jednotlivého assetu, načítání poslední telemetrie, načítání historie telemetrie, výpis všech alertů, výpis alertů pro konkrétní asset a endpoint `/metrics` pro Prometheus.

Pro odpovědi je použito streamové kódování přes `json.NewEncoder()`, kterým je eliminována alokace mezipaměti pro celý JSON dokument. U endpointu pro historii telemetrie byl implementován limit s výchozí hodnotou 150 záznamů a maximální hodnotou 1000 záznamů, čímž je předejito vyčerpání paměti při příliš velkém dotazu.

5.6 Simulator

Simulátor je samostatná binárka, jejíž veškerá logika je obsažena v souboru `backend/cmd/simulator/main.go`. V rámci funkce `main()` je nejprve inicializován `OpenTelemetry` tracer provider a vytvořen gRPC klient k `Ingest` službě. Následně je v cyklu spuštěno 50 goroutin, z nichž každá reprezentuje jeden lodní kontejner a každou sekundu odesílá jeden telemetrický záznam.

Pro výpočet aktuální polohy lodi je definováno 10 GPS waypointů na trase Pusan, Rotterdam. Lineární interpolací mezi sousedními waypointy je získána aktuální pozice, přičemž rychlost lodi je nastavena na 15 uzlů a simulace je zrychlena dvacetinásobně. Celková délka trasy činí přibližně 18 734 km (10 116 námořních mil), reálná plavba by trvala přibližně 28 dní, po dvacetinásobném zrychlení je tak celá plavba modelována na přibližně 34 hodin simulovaného reálného času. Teplota je modelována denním sinusovým cyklem s bází mezi 14 a 26 °C, vlhkost dvanáctihodinovým sinusovým cyklem s bází mezi 30 a 40 % RH. K oběma hodnotám je dále přičten náhodný šum a per-kontejnerový offset, kterým jsou modelovány drobné rozdíly mezi jednotlivými umístěními v nákladovém prostoru.

5.7 Klientská aplikace

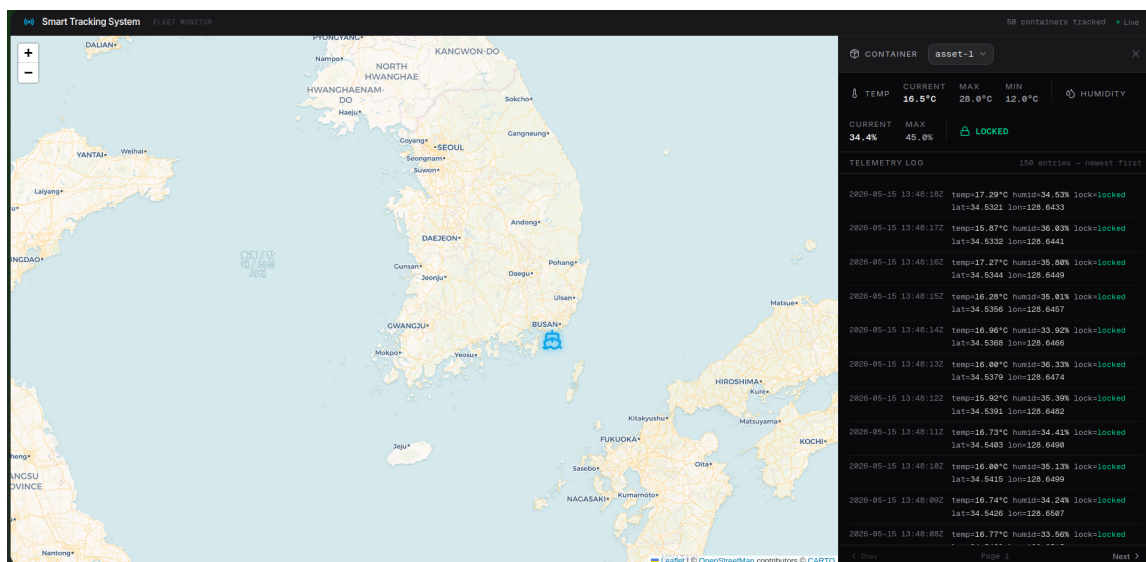
Klientská aplikace je tvořena single-page Next.js aplikací, jejíž kořenová stránka je definována v souboru `frontend/app/page.tsx`. Hlavní část obrazovky je věnována interaktivní mapě s polohou lodi, která je vykreslována komponentou `ShipMap` (soubor `frontend/components/ShipMap.tsx`). Mapa je založena na knihovně Leaflet, která je dynamicky načítána z veřejného CDN při prvním vykreslení komponenty. Pro každou unikátní GPS pozici je na mapě umístěn značka v podobě SVG ikony lodi, přičemž po kliknutí na značku je v pravém panelu zobrazen detail kontejneru.

Listing 5 ukazuje použití knihovny SWR pro periodické stahování dat z REST API.

```
1 "use client";
2
3 import useSWR from "swr";
4 import { fetchAssets, fetchLastTelemetry } from "@lib/api";
5 import type { Asset, Telemetry } from "@lib/schemas";
6
7 export function useAssets() {
8   return useSWR<Asset[]>("assets", fetchAssets, { refreshInterval: 10_000 });
9 }
10
11 export function useLastTelemetry(assetId: string | null) {
12   return useSWR<Telemetry>({
13     assetId ? `telemetry/${assetId}` : null,
14     () => fetchLastTelemetry(assetId!),
15     { refreshInterval: 5_000 }
16   });
17 }
```

Listing 5: SWR hooky pro pravidelné stahování dat (`frontend/hooks/useAssets.ts`)

Veškeré odpovědi z REST API jsou validovány knihovnou Zod, jejíž schémata jsou v souboru `frontend/lib/schemas.ts`. Tím je zajištěno, že případná nekonzistence mezi backendovým modelem a frontendovým očekáváním je odhalena již při běhu klienta, a nikoliv až v průběhu vykreslení komponenty. Náhled klientské aplikace je k vidění na Obrázku 1.



Obrázek 1: Klientská dashboard aplikace s mapou pozice lodi a panelem telemetrie

6 Nasazení a infrastruktura

V této kapitole je popsán postup nasazení celého systému do k3d clusteru, počínaje deklarativním provisionováním pomocí Terraformu, přes konfiguraci ArgoCD pro GitOps, až po definici CI pipeline v Tektonu.

6.1 Infrastruktura jako kód s Terraformem

Terraform soubory v adresáři `backend/deploy/terraform/` obsahují kompletní popis infrastruktury, která je vytvořena spuštěním jediného příkazu `terraform apply`. V souboru `main.tf` jsou deklarováni tři Terraform provideri, `moio/k3d`, `hashicorp/kubernetes` a `hashicorp/helm`, a je provedena jejich konfigurace s přihlašovacími údaji clusteru čerpanými přímo ze zdroje `k3d_cluster`, čímž je eliminována nutnost dvoufázového nasazení.

V souboru `cluster.tf` je definován k3d cluster s jedním control-plane uzlem, dvěma worker uzly a mapováním portu 8080 hostitelského počítače na port 80 uvnitř clusteru, čímž je zpřístupněn Traefik Ingress. V souboru `namespaces.tf` jsou vytvořeny čtyři namespace, `infrastructure`, `tracking-system`, `argocd` a `monitoring`. V souboru `infrastructure.tf` jsou pak instalovány Helm releases, kterými jsou nasazeny NATS JetStream, PostgreSQL (Bitnami chart), kube-prometheus-stack, Grafana Tempo a Loki-stack. Příklad jednoho takového releasu je v Listingu 6.

```
1 resource "helm_release" "nats" {
2     name           = var.helm_release_nats_name
3     repository     = var.helm_release_nats_repo
4     chart          = var.helm_release_nats_chart
5     namespace      = kubernetes_namespace.infra.metadata[0].name
6
7     set = [
8         {
9             name = "config.jetstream.enabled"
10            value = "true"
11        },
12        {
13            name = "config.cluster.enabled"
14            value = "false"
15        }
16    ]
17 }
```

Listing 6: Definice Helm releasu pro NATS JetStream (`infrastructure.tf`)

6.2 Kubernetes manifesty

Veškeré aplikační manifesty jsou umístěny v adresáři `backend/deploy/k8s/`, kde je ArgoCD sleduje a automaticky aplikuje při každé změně. Pro každou ze čtyř mikroslužeb je definován jeden Deployment a jeden Service objekt. Výjimkou je služba `simulator`, ke které není přidružen

žádný Service, jelikož je pouze odchozím klientem. Naopak služba **ingest** má v Service objektu vystaveny dva porty, 50051 pro gRPC a 9090 pro Prometheus metriky.

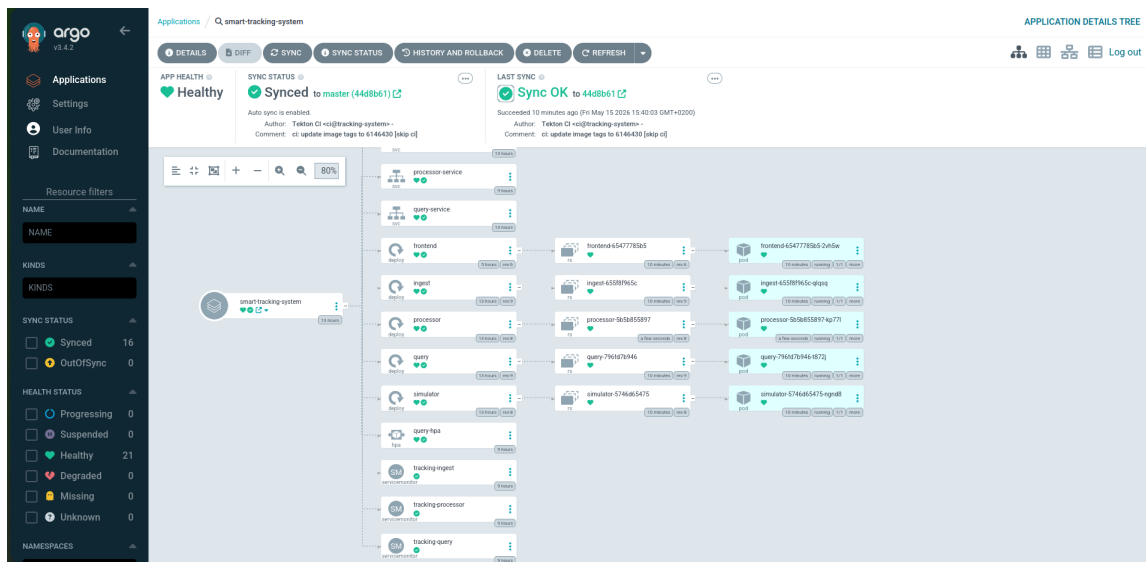
Pro směrování externího provozu byl vytvořen Ingress objekt (soubor **ingress.yaml**), který je obsluhován Traefikem. Prefixy cest **/assets** a **/alerts** jsou směrovány do služby **query**, veškerý ostatní provoz pak do služby **frontend**. Pro službu **query** je definován HorizontalPodAutoscaler (soubor **hpa.yaml**), kterým je počet replik škálován mezi jednou a třemi instancemi v závislosti na 70% využití CPU. Sdílená nekonfidenční konfigurace je obsažena v ConfigMap **tracking-config** (soubor **configmap.yaml**), citlivé hodnoty jako URL databáze jsou v Secret **tracking-secret** (soubor **secret.yaml**), který je v repozitáři ignorován a musí být na každém clusteru vytvořen ručně.

Schéma databáze a počáteční seed dat 50 assetů je vytvářen Kubernetes Jobem **db-init**, který je opatřen anotací **argocd.argoproj.io/hook: PreSync**. Tím je zajištěno, že databázové schéma je k dispozici dříve, než jsou startovány aplikační pody.

6.3 GitOps s ArgoCD

ArgoCD je nakonfigurován jako jediný zdroj pravdy pro stav clusteru. Po naklonování repozitáře a přihlášení do ArgoCD UI je vytvořena Application, která je nakonfigurována tak, aby sledovala adresář **backend/deploy/k8s** na větvi **master** a aplikovala změny s politikou **Automatic** a **selfHeal: true**. ArgoCD periodicky (každé 3 minuty) kontroluje Git repozitář a v případě odchylky aplikuje aktualizované manifesty.

Sekvence událostí při běžné iteraci vývoje je následující. Vývojářem je commitnuta změna v Go kódu na větev **master**. Manuálně je spuštěn Tekton PipelineRun, kterým jsou sestaveny nové Docker obrazy, jsou pushnuty do GitHub Container Registry s tagem ve formě krátkého Git SHA, a následně je commitnut do repozitáře aktualizovaný image tag v odpovídajících deployment manifestech. Tato Git změna je následně detekována ArgoCD, které provede synchronizaci stavu a aktualizované pody jsou postupně rolled out. Pro okamžitou aplikaci bez čekání na pravidelný polling je k dispozici příkaz pro vynucený hard refresh. Webové rozhraní ArgoCD se stavem nasazení je zachyceno na Obrázku 2.



Obrázek 2: Webové rozhraní ArgoCD se stavem synchronizace aplikace

6.4 Tekton pipeline

Tekton CI pipeline je definována v souboru `backend/deploy/tekton/pipeline.yaml`. Pipeline obsahuje devět kroků, které jsou spouštěny sekvenčně. Prvním krokem je naklonování repozitáře (`git-clone`), druhým spuštění `go test ./...` pro veškeré balíčky (`run-tests`). Pokud testy selžou, je pipeline okamžitě zastavena a žádný obraz není sestaven. Tím je zaručeno, že v GHCR existují pouze obrazy z kódu, který prošel testy.

Následuje pět kroků sestavení a push obrazů (`build-push-ingest`, `build-push-processor`, `build-push-query`, `build-push-simulator`, `build-push-frontend`), které jsou v současné konfiguraci spouštěny sekvenčně. Tímto je předejito vyčerpání DNS pod zátěží paralelních Kaniko buildů. Každý obraz je tagován dvěma tagy, krátkým Git SHA pro neměnnou identifikaci a `latest` pro pohodlnost. Posledním krokem je `update-manifest`, ve kterém je pomocí `sed` přepsán image tag ve všech pěti deployment manifestech a je proveden commit a push zpět do Git repozitáře. Tím je uzavřena smyčka mezi CI pipeline a GitOps nasazením. Ukázka definice tasku pro build a push je v Listingu 7.

```

1 apiVersion: tekton.dev/v1
2 kind: Task
3 metadata:
4   name: build-push
5   namespace: tracking-system
6 spec:
7   params:
8     - name: service
9     - name: image-tag
10    - name: registry
11    default: ghcr.io/bojkoj

```

```
12     workspaces:
13         - name: source
14         - name: docker-credentials
15           mountPath: /kaniko/.docker
16     steps:
17         - name: build-and-push
18           image: gcr.io/kaniko-project/executor:latest
19           args:
20             - --dockerfile=$(workspaces.source.path)/backend/Dockerfile
21             - --context=$(workspaces.source.path)/backend
22             - --build-arg=SERVICE=$(params.service)
23             - --destination=$(params.registry)/$(params.service):$(params.
image-tag)
24             - --destination=$(params.registry)/$(params.service):latest
25             - --cache=true
26             - --cache-repo=$(params.registry)/kaniko-cache
```

Listing 7: Tekton Task pro build a push obrazu pomocí Kaniko (task-build-push.yaml)

6.5 Vícefázový Dockerfile

Pro všechny čtyři Go mikroslužby je sdílen jeden vícefázový Dockerfile v souboru `backend/Dockerfile`. V první fázi (`builder`) je z obrazu `golang:1.25-alpine` zkompileována Go binárka pro konkrétní službu, která je vybrána parametrem `ARG SERVICE`. V druhé fázi (`runner`) je z minimálního obrazu `alpine:3.19` zkopírována pouze výsledná binárka, čímž je dosaženo cílové velikosti přibližně 20 MB. Pro běh procesu byl vytvořen dedikovaný non-root uživatel `appuser`, kterým je dodržena bezpečnostní praktika minimálního oprávnění.

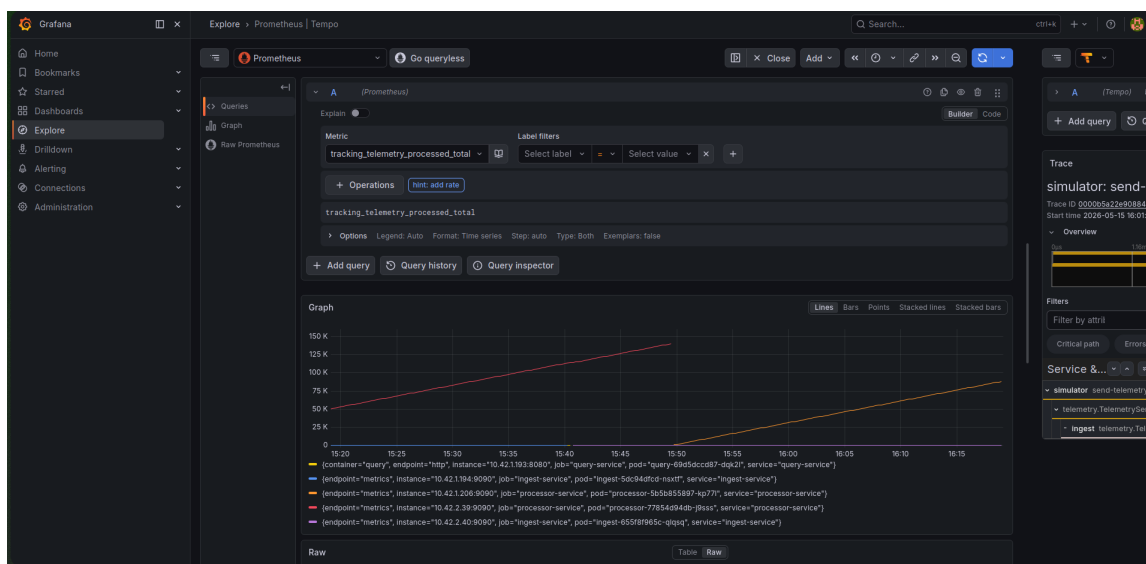
7 Observabilita

Observabilita systému byla v projektu řešena podle modelu tří pilířů, kterými jsou metriky, distribuované trasování a logy. Pro každý z těchto pilířů je v namespace `monitoring` nasazena samostatná komponenta a veškerá data jsou unifikovaně zobrazitelná v Grafaně.

7.1 Metriky

Aplikační metriky jsou definovány v souboru `backend/internal/common/metrics.go` a jsou zaregistrovány do default Prometheus registry pomocí knihovny `promauto`. Konkrétně byly zavedeny čtyři metriky. `tracking_ingest_requests_total` typu `CounterVec` měří počet gRPC volání přijatých službou `ingest` a je labellována dle pole `success`. `tracking_telemetry_processed_total` typu `Counter` měří počet zpráv zpracovaných službou `processor`. `tracking_alerts_generated_total` typu `CounterVec` měří počet vygenerovaných alertů dle typu. `tracking_processing_duration_seconds` typu `Histogram` měří latenci zpracování jedné telemetrické zprávy.

Pro každou ze tří služeb (`ingest`, `processor`, `query`) byl vytvořen `ServiceMonitor` objekt v souboru `servicemonitor.yaml`, který je opatřen labellem `release: kube-prometheus-stack`. Tímto je Prometheus Operátorem automaticky rozpoznán a jednotlivé služby jsou scrapovány každých 15 sekund. Náhled vlastních metrik vizualizovaných v Grafaně je k vidění na Obrázku 3.



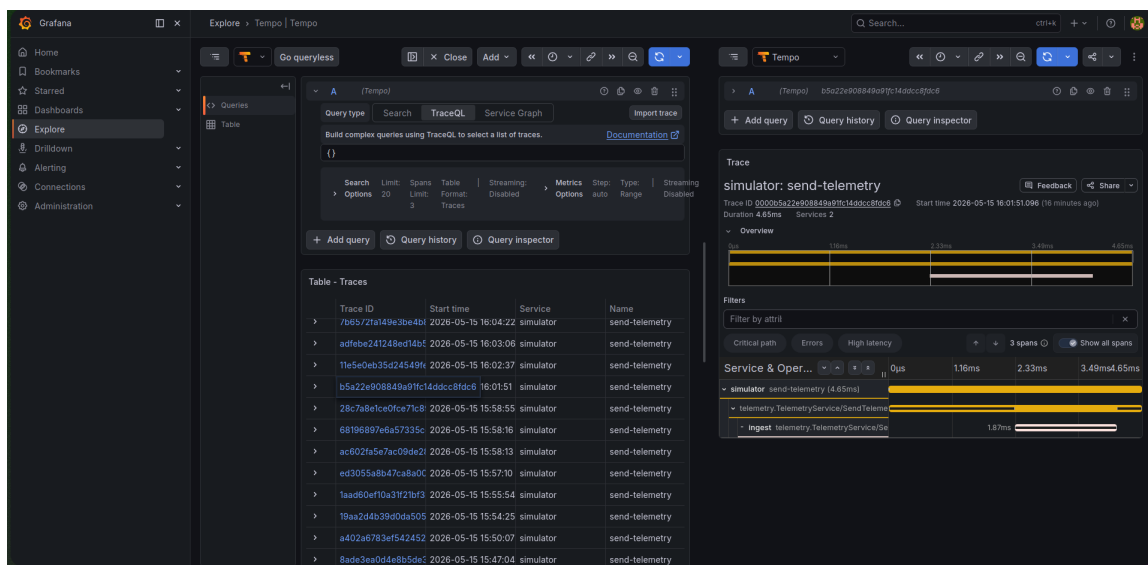
Obrázek 3: Vlastní aplikační metriky vizualizované v Grafaně

7.2 Distribuované trasování

Distribuované trasování je řešeno pomocí OpenTelemetry SDK. Inicializace je provedena funkcí `InitTracerProvider()` v souboru `backend/internal/common/otel.go`. Pokud je nastavena

proměnná prostředí `OTEL_EXPORTER_OTLP_ENDPOINT`, jsou trace spany exportovány přes OTLP gRPC do Grafana Tempa. V opačném případě jsou spany vypisovány na standardní výstup, čímž je usnadněn lokální vývoj.

Propagace trace kontextu je zajištěna standardem W3C TraceContext a probíhá přes dvě hranice. Mezi Simulátorem a Ingestem je kontext propagován prostřednictvím `otelgrpc` stats handleru, kterým jsou injektovány hlavičky `traceparent` a `tracestate` do gRPC metadat. Mezi Ingestem a Processorem je kontext propagován prostřednictvím vlastní implementace `TextMapCarrier` v NATS hlavičkách. Tím je v Tempu zachována souvislá trace přes celý pipeline, počínaje generováním telemetrie ve Simulatoru a konče uložením do PostgreSQL Processorem. Ukázka detailu trace v Grafana Tempu je na Obrázku 4.



Obrázek 4: Distribuovaná trace zachycena v Grafana Tempu

7.3 Logy

Logy jsou ve všech čtyřech Go službách generovány standardní knihovnou `log/slog` ve strukturovaném JSON formátu. Logger je inicializován funkcí `NewLogger()` v souboru `backend/internal/common/logger.go`, kterou je každému záznamu automaticky přidán atribut `service` s jménem dané mikroslužby. Tím je v Loki umožněn jednoduchý filtr podle pole `service`, čímž jsou logy efektivně oddělitelné mezi službami.

Promtail běží jako DaemonSet na každém uzlu clusteru a čte logy z adresáře `/var/log/pods`. Tyto logy jsou indexovány v Loki a v Grafaně mohou být filtrovány například výrazem `{namespace="tracking-system", service="processor"}`.

8 Testování

Pro ověření funkcionality byla implementována sada 72 automatizovaných unit testů. Testy jsou napsány ve standardním Go testing frameworku bez jakékoliv externí závislosti. Pro testování HTTP handlerů je využit balíček `net/http/httptest`, kterým je v rámci procesu spuštěn dočasný HTTP server. Pro izolaci od skutečné infrastruktury jsou definovány mock struktury implementující portová rozhraní `AssetRepository`, `TelemetryRepository`, `AlertRepository` a `EventProducer`.

Rozložení testů je následující. Soubor `http_handler_test.go` obsahuje 15 testů s pokrytím 87,8% pro handler balíček. Soubor `grpc_handler_test.go` obsahuje 5 testů. Soubor `processing_test.go` obsahuje 14 testů s pokrytím 89,2%. Soubor `ingestion_test.go` obsahuje 12 testů. Soubor `main_test.go` u Simulatoru obsahuje 18 testů, které pokrývají Haversinovu funkci, výpočet pozice lodi, výpočet teploty a výpočet vlhkosti. Soubor `config_test.go` pak obsahuje 8 testů pro načítání konfigurace z proměnných prostředí.

Testy jsou navrženy jako zaručovatelé kvality v CI pipeline. Jsou spouštěny Tektonem jako druhý krok pipeline, hned po naklonování repozitáře a před jakýmkoliv Docker buildem. Pokud testy selžou, je celá pipeline okamžitě zastavena a žádný nový obraz není sestaven ani pushnut. Tím je zaručeno, že v GitHub Container Registry je dostupný pouze kód, který prošel testy.

9 Závěr

V rámci tohoto semestrálního projektu byl navržen a implementován funkční prototyp distribuovaného systému pro správu firemních aktiv, který demonstruje praktické uplatnění Cloud-Native principů. Cíle stanovené v zadání byly splněny, byl realizován backend tvořený čtyřmi Go mikroslužbami komunikujícími přes gRPC a NATS JetStream, klientská dashboard aplikace postavená na frameworku Next.js, kompletní infrastruktura definovaná pomocí Terraformu a Helm chartů, GitOps nasazení s ArgoCD a automatizovaná CI pipeline v Tektonu. Byla zajištěna observabilita ve všech třech pilířích pomocí OpenTelemetry, Prometheus, Grafana, Tempo a Loki a byla vytvořena sada 72 automatizovaných testů, které jsou součástí CI pipeline.

V rámci implementace byly aplikovány návrhové vzory Clean Architecture, Event-Driven Architecture, CQRS a Dependency Injection. Touto kombinací bylo dosaženo systému, ve kterém je doménová vrstva oddělena od konkrétních infrastrukturních detailů, čtecí a zápisové cesty jsou nezávisle škálovatelné, a kód je dobře testovatelný díky portovým rozhraním a mockovatelným závislostem.

Mezi možná rozšíření, která by mohla být v budoucnu implementována, lze zařadit přidání paralelního zpracování v Processoru pomocí pull-based JetStream konzumentů s více workery, nasazení horizontálního autoškálování i pro službu `ingest` na základě metriky příchozích gRPC volání za sekundu, integraci s externí mapovou službou pro lepší vizualizaci námořních tras a nasazení komponenty AlertManager pro odesílání notifikací o překročení limitů prostřednictvím e-mailu nebo Slacku.

Reference

1. Cloud Native Computing Foundation. CNCF Cloud Native Definition. Dostupné z: <https://github.com/cncf/toc/blob/main/DEFINITION.md>
2. Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.
3. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
4. Fowler, M. What do you mean by Event-Driven? Dostupné z: <https://martinfowler.com/articles/201701-event-driven.html>
5. Fowler, M. CQRS. Dostupné z: <https://martinfowler.com/bliki/CQRS.html>
6. Wiggins, A. The Twelve-Factor App. Dostupné z: <https://12factor.net/>
7. Donovan, A. A., & Kernighan, B. W. (2015). *The Go Programming Language*. Addison-Wesley.
8. gRPC oficiální dokumentace. Dostupné z: <https://grpc.io/docs/>
9. NATS oficiální dokumentace. Dostupné z: <https://docs.nats.io/>
10. Next.js dokumentace. Dostupné z: <https://nextjs.org/docs>
11. Luksa, M. (2017). *Kubernetes in Action*. Manning Publications.
12. Tekton Pipelines dokumentace. Dostupné z: <https://tekton.dev/docs/pipelines/>
13. OpenTelemetry oficiální dokumentace. Dostupné z: <https://opentelemetry.io/docs/>
14. PostgreSQL oficiální dokumentace. Dostupné z: <https://www.postgresql.org/docs/>
15. pgx PostgreSQL driver pro Go. Dostupné z: <https://github.com/jackc/pgx>
16. Helm oficiální dokumentace. Dostupné z: <https://helm.sh/docs/>
17. Terraform oficiální dokumentace. Dostupné z: <https://developer.hashicorp.com/terraform/docs>
18. ArgoCD oficiální dokumentace. Dostupné z: <https://argo-cd.readthedocs.io/>
19. k3s odlehčená distribuce Kubernetes. Dostupné z: <https://k3s.io/>
20. k3d nástroj pro k3s v Dockeru. Dostupné z: <https://k3d.io/>
21. Kaniko nástroj pro build kontejnerů v Kubernetes. Dostupné z: <https://github.com/GoogleContainerTools/kaniko>
22. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.